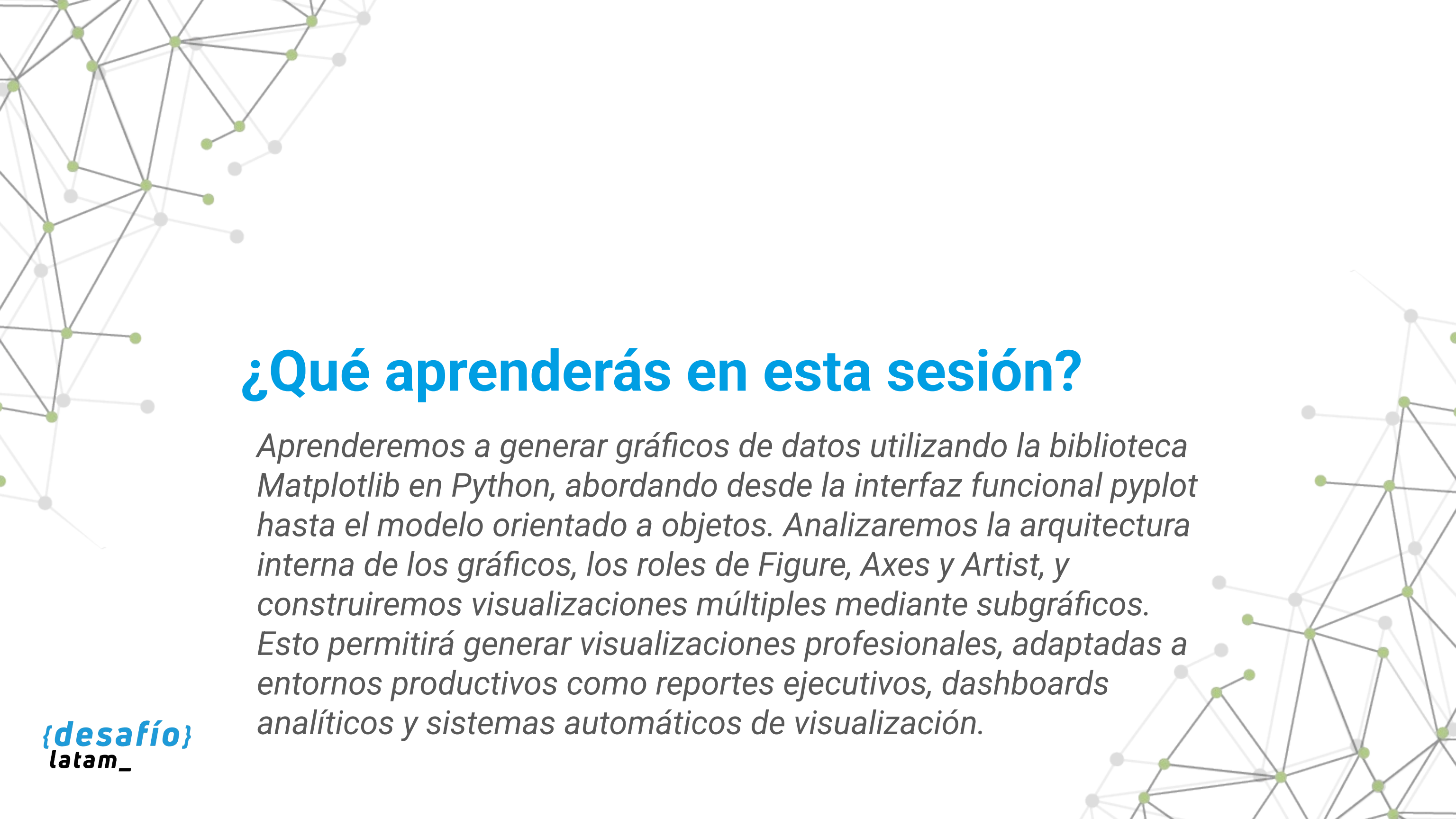




Visualización de datos

Clase 6 - Visualización de datos con Matplotlib



¿Qué aprenderás en esta sesión?

Aprenderemos a generar gráficos de datos utilizando la biblioteca Matplotlib en Python, abordando desde la interfaz funcional pyplot hasta el modelo orientado a objetos. Analizaremos la arquitectura interna de los gráficos, los roles de Figure, Axes y Artist, y construiremos visualizaciones múltiples mediante subgráficos. Esto permitirá generar visualizaciones profesionales, adaptadas a entornos productivos como reportes ejecutivos, dashboards analíticos y sistemas automáticos de visualización.

Desafío inicial

El problema

Te encuentras desarrollando un dashboard exploratorio de ventas para una empresa de retail. La dirección comercial necesita visualizar comparativamente los volúmenes de ventas por trimestre y región, junto con la evolución de las ventas mensuales. Se espera que generes un conjunto de gráficos coherentes y organizados en una sola figura, permitiendo su exportación como imagen para informes.

El desafío

Sin embargo, para cumplir este requerimiento, necesitas dominar las herramientas que ofrece Matplotlib, desde el uso rápido de pyplot, hasta el control preciso que entrega el modelo orientado a objetos, así como la capacidad de trabajar con subgráficos. ¿Cómo lo harás?

`/* Funciones pyplot */`

Crear un gráfico con las funciones pyplot

¿Cómo visualizar rápidamente patrones o tendencias en un conjunto de datos?

En etapas tempranas del análisis de datos, es habitual necesitar una representación gráfica básica que nos permita explorar el comportamiento general de nuestras variables. Para esta tarea, la biblioteca Matplotlib en Python ofrece una interfaz funcional conocida como pyplot, que permite construir gráficos de forma secuencial con muy pocas líneas de código.

El uso de pyplot es especialmente útil en contextos como prototipado rápido, notebooks interactivos (Jupyter, Google Colab) o scripts de validación, donde la velocidad de desarrollo prima sobre la complejidad estructural. No obstante, su uso adecuado requiere comprender cómo Matplotlib gestiona internamente los elementos visuales.



Fundamentos de pyplot

¿Cómo funciona?

pyplot funciona como una "pizarra activa": cada vez que llamamos a una función como `plot()` o `title()`, Matplotlib va añadiendo elementos a una figura implícita que se mantiene en memoria. Esta figura sólo se muestra cuando se ejecuta `plt.show()`.

Esto implica que:

- Las funciones actúan sobre una figura y un eje preexistentes, sin que el usuario los cree explícitamente.
- Si no se gestiona correctamente, pueden quedar residuos gráficos de ejecuciones anteriores (por ejemplo, si se ejecutan varias celdas sin limpiar el gráfico).

Arquitectura básica de un gráfico con pyplot

Cada gráfico construido con pyplot se compone al menos de los siguientes elementos:

Componente	Descripción
Figura (figure)	El lienzo o espacio donde se dibuja el gráfico.
Ejes (axes)	Área dentro de la figura donde se representa la visualización.
Datos	Serie de puntos, líneas o barras a representar.
Metadatos	Título, etiquetas de ejes, leyendas, grillas.

Aunque no definimos estos elementos directamente, pyplot los crea en segundo plano y los va completando con cada instrucción.

Ejemplo:

Supongamos que estamos analizando las ventas mensuales del primer cuatrimestre de una tienda:

```
import matplotlib.pyplot as plt
meses = ['Ene', 'Feb', 'Mar', 'Abr']
ventas = [300, 320, 310, 330]
# Crear figura y grafico
plt.figure(figsize=(8, 5))
# Definimos tamaño en pulgadas
plt.plot(meses, ventas, color='steelblue', marker='o',
linestyle='--', linewidth=2)
# Agregar metadatos
plt.title('Ventas mensuales primer cuatrimestre')
plt.xlabel('Mes')
plt.ylabel('Ventas (miles de $)')
plt.grid(True)
# Mostrar la figura
plt.show()
```



MONTHLY SALES DALES



Matplotlib Parameters Visualization

Explicaciones clave:



Marcadores

`marker='o'`: permite visualizar cada punto individual.



Estilo de línea

`linestyle='--'`: agrega estilo discontinuo a la línea.



Grosor

`linewidth`: controla el grosor del trazo.

Este gráfico entrega una lectura clara de la evolución mensual de las ventas, destacando la tendencia ascendente.

Tipos de gráficos disponibles con pyplot

Matplotlib incluye una gran variedad de funciones específicas para representar diferentes tipos de datos. Algunas de las más usadas son:

Función	Uso típico
plot()	Series temporales, líneas
bar(), barh()	Comparaciones categóricas
scatter()	Relaciones entre dos variables
hist()	Distribuciones de frecuencia
pie()	Proporciones (con reservas de uso)

Cada una de estas funciones comparte parámetros comunes como color, tamaño de letra, etiquetas, etc., lo que permite mantener un estilo coherente en los gráficos.

Buenas prácticas

En el desarrollo de visualizaciones profesionales, es crucial adoptar ciertas prácticas que aseguren claridad, coherencia y robustez en la representación de datos. A continuación, se describen recomendaciones fundamentales al trabajar con pyplot:

Usar `plt.figure(figsize=(ancho, alto))`: Mejora la legibilidad al controlar el espacio físico que ocupará el gráfico, sobre todo si se va a exportar.

Incluir etiquetas y título: Un gráfico sin contexto no transmite información. Al menos se debe incluir título, etiquetas de ejes y grilla.

Ejecutar `plt.show()` una sola vez al final: Evita renderizados múltiples que confunden o ralentizan el entorno.

Usar `plt.clf()` si se requiere limpiar el estado: Útil cuando se generan muchos gráficos secuenciales en un mismo script.

Errores comunes y cómo evitarlos

Aunque la sintaxis de pyplot es sencilla, su uso inadecuado puede provocar problemas que afectan la claridad o funcionalidad de los gráficos.

Estos errores, si bien frecuentes, pueden prevenirse con algunas medidas simples:

Error	Causa probable	Solución recomendada
El gráfico no se muestra	Olvido de plt.show()	Incluirlo al final del bloque de código.
Los gráficos se superponen al ejecutar varias veces	Estado persistente de la figura implícita	Usar plt.clf() o plt.close('all') entre ejecuciones.
Texto ilegible o cortado	Tamaño por defecto muy pequeño	Usar figsize y evitar etiquetas largas.

Limitaciones de pyplot

Identificar estos errores y aplicar las soluciones recomendadas no solo mejora la calidad de las visualizaciones, sino que permite escalar los códigos a entornos más exigentes como dashboards, informes automáticos o presentaciones analíticas.

Aunque pyplot es ideal para gráficos rápidos y sencillos, su alcance es limitado cuando buscamos mayor control y personalización.

Para abordar estos desafíos, pasaremos ahora al modelo de orientación a objetos de Matplotlib, que nos ofrece una forma más estructurada y potente de construir visualizaciones.



Modelo de orientación a objetos (OO) de Matplotlib

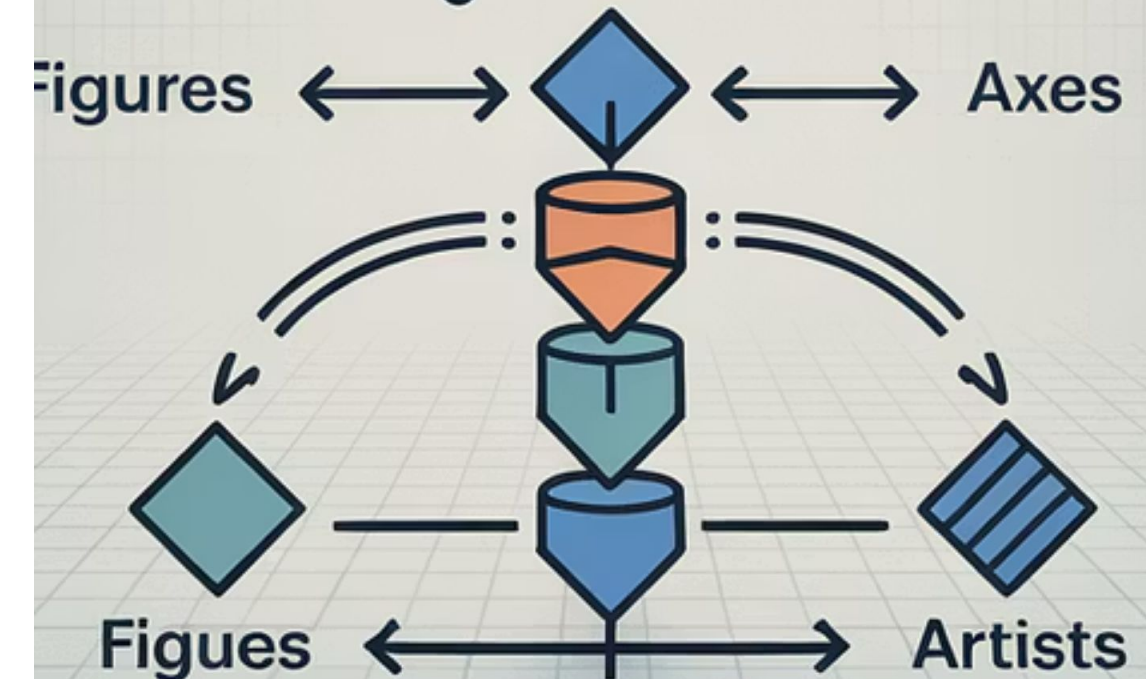
¿Cómo podemos estructurar y personalizar un gráfico para lograr mayor flexibilidad en visualizaciones complejas?

El modelo de orientación a objetos (OO) de Matplotlib ofrece una forma más controlada, modular y escalable para crear gráficos. A diferencia de pyplot, que oculta la arquitectura interna y manipula una figura implícita, el modelo OO permite acceder directamente a los objetos gráficos como Figure y Axes, facilitando una personalización precisa de cada componente.

Este enfoque es especialmente útil en proyectos donde se requiere:

- Crear gráficos automáticos desde funciones o scripts reutilizables.
- Integrar visualizaciones en informes profesionales o dashboards.
- Componer gráficos múltiples o anidados en una misma figura.

Matplotlib



Arquitectura general

La estructura básica de Matplotlib en su enfoque OO es jerárquica y se compone de:

Componente	Descripción
Figure	Lienzo general donde se dibujan uno o varios gráficos.
Axes	Recuadro o área dentro de la figura donde se representa el gráfico. Cada Axes tiene su propio sistema de coordenadas.
Axis	Cada uno de los ejes X o Y en un Axes. Controla los valores, tics y etiquetas.
Artist	Cualquier objeto visible: títulos, textos, líneas, cuadros, leyendas, etc.

Creando un gráfico con el modelo OO

Para trabajar con esta arquitectura, se utiliza la función `plt.subplots()`, que devuelve una tupla con la figura y los ejes:

```
import matplotlib.pyplot as plt
# Datos
meses = ['Ene', 'Feb', 'Mar', 'Abr']
ventas = [300, 320, 310, 330]
# Crear la figura y los ejes
fig, ax = plt.subplots(figsize=(8, 5))
# Dibujar en los ejes
ax.plot(meses, ventas, color='darkgreen', marker='o')
# Personalizar elementos
ax.set_title('Evolución de ventas primer cuatrimestre')
ax.set_xlabel('Meses')
ax.set_ylabel('Miles de $')
ax.grid(True)
# Mostrar
plt.show()
```



Explicación del modelo OO

Aquí fig representa el lienzo general, mientras que ax es el área donde se dibuja el gráfico. A diferencia de pyplot, todos los elementos se agregan directamente al objeto ax, lo que mejora la trazabilidad del código.

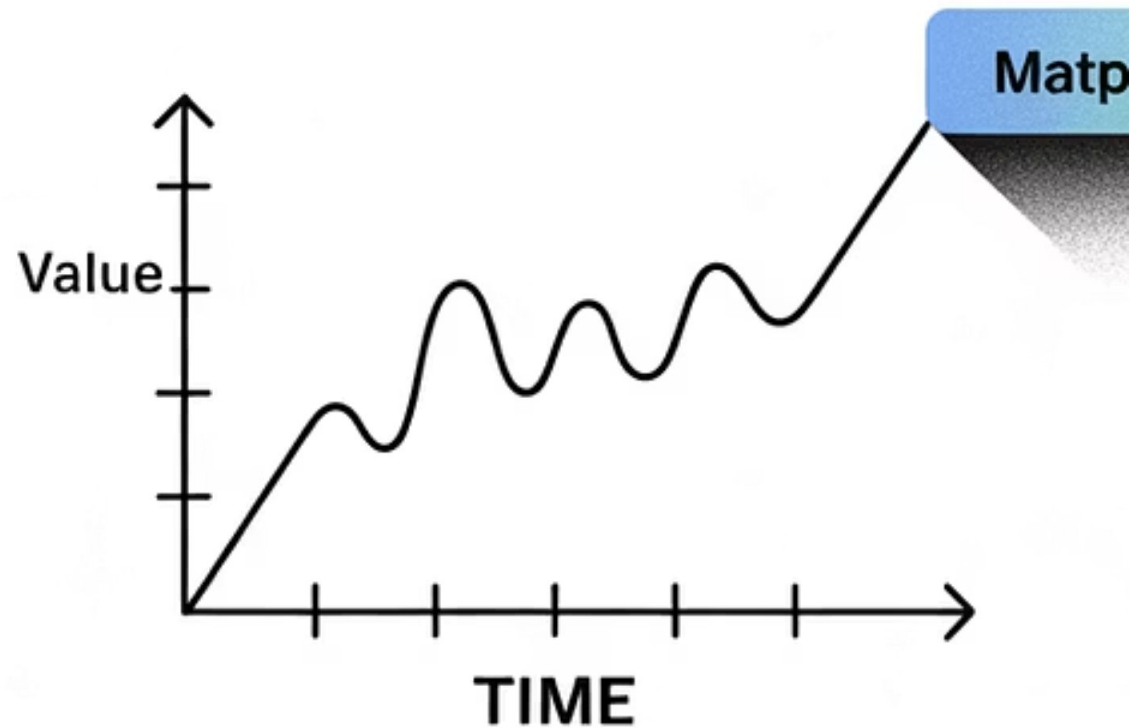


Figure (fig)

El lienzo general donde se dibuja todo



Axes (ax)

El área específica del gráfico



Métodos de ax

Controlan cada aspecto del gráfico

Ventajas clave del modelo 00

Reutilización

Se pueden encapsular visualizaciones dentro de funciones.

Composición

Es posible organizar varios gráficos en una misma figura.

Control preciso

Permite modificar componentes específicos como escalas, anotaciones, formatos de tics, etc.



Comparación técnica

Para tomar decisiones fundamentadas sobre qué enfoque utilizar, es importante comparar el modelo funcional (pyplot) con el modelo orientado a objetos (OO) desde una perspectiva técnica y operativa. La siguiente tabla resume las principales diferencias entre ambos enfoques, considerando criterios como la modularidad, escalabilidad y aplicabilidad en proyectos complejos.

Característica	pyplot (funcional)	Modelo OO (estructurado)
Figura implícita	Sí	No (requiere definición)
Modularidad	Baja	Alta
Escalabilidad	Limitada	Completa
Uso en proyectos complejos	No recomendado	Recomendado

Ejemplo aplicado: función reutilizable

Una ventaja clave del modelo OO es la posibilidad de definir funciones que generen gráficos sin depender del estado global:

```
def grafico_ventas(meses, ventas, titulo):  
    fig, ax = plt.subplots(figsize=(7, 4))  
    ax.plot(meses, ventas, marker='s', color='teal')  
    ax.set_title(titulo)  
    ax.set_xlabel('Meses')  
    ax.set_ylabel('Ventas ($)')  
    ax.grid(True)  
    return fig  
  
# Llamado de la función  
grafico_ventas(['Ene', 'Feb', 'Mar'], [300, 280, 320], 'Ventas  
Trimestre 1')  
plt.show()
```

Este enfoque es útil cuando se generan visualizaciones repetitivas, como parte de un reporte mensual o en un sistema de alertas automáticas.

Visualize your code

Implement best practices for efficient and readable data visualizations

Get Started



Buenas prácticas con el modelo OO

Nombrar las figuras y ejes con nombres descriptivos: mejora la legibilidad y facilita el debugging.

Separar la definición de datos del código gráfico: ayuda a desacoplar lógica y visualización.

Encapsular visualizaciones dentro de funciones: fundamental para mantener el código limpio y reutilizable.

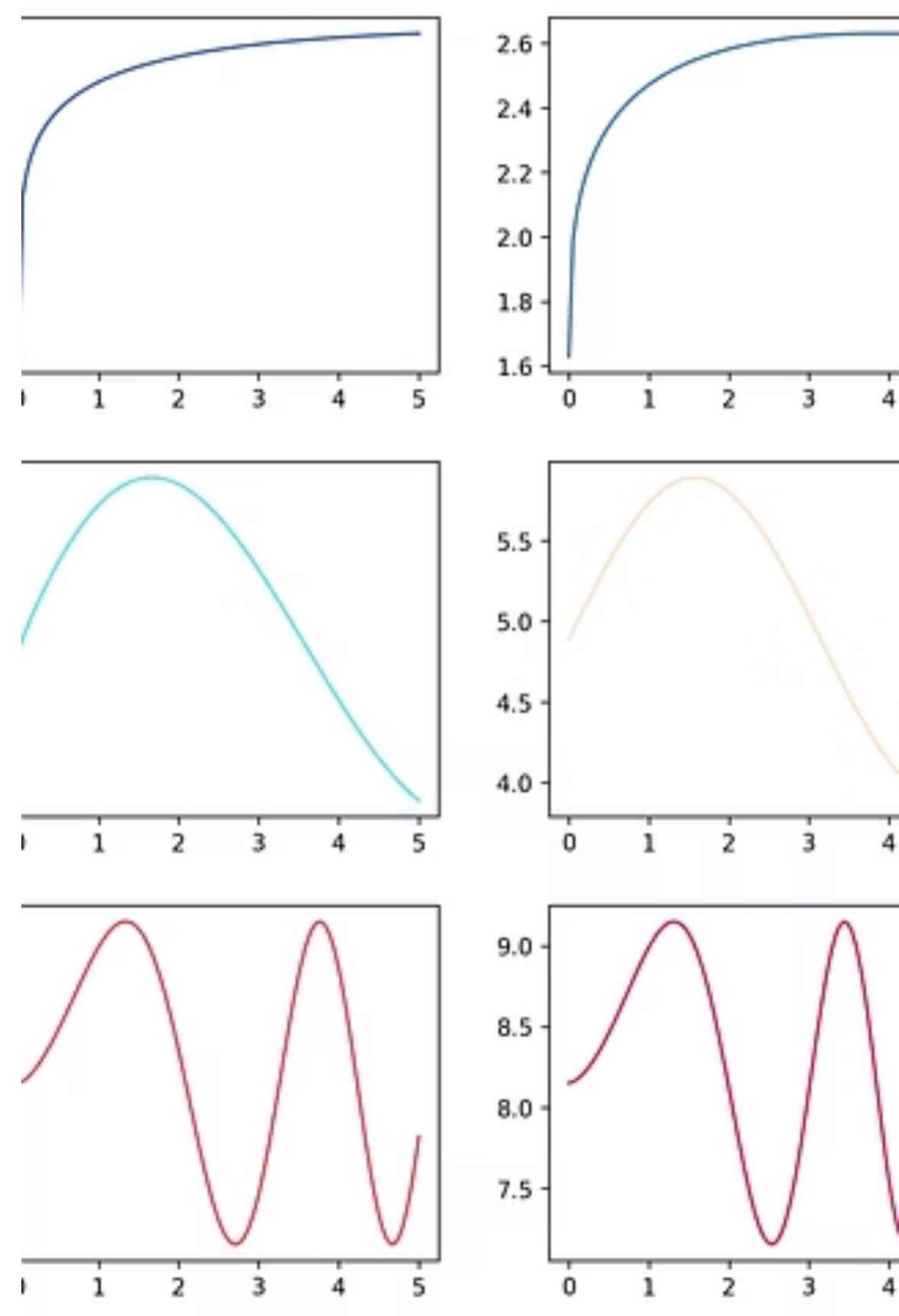
Errores comunes y recomendaciones

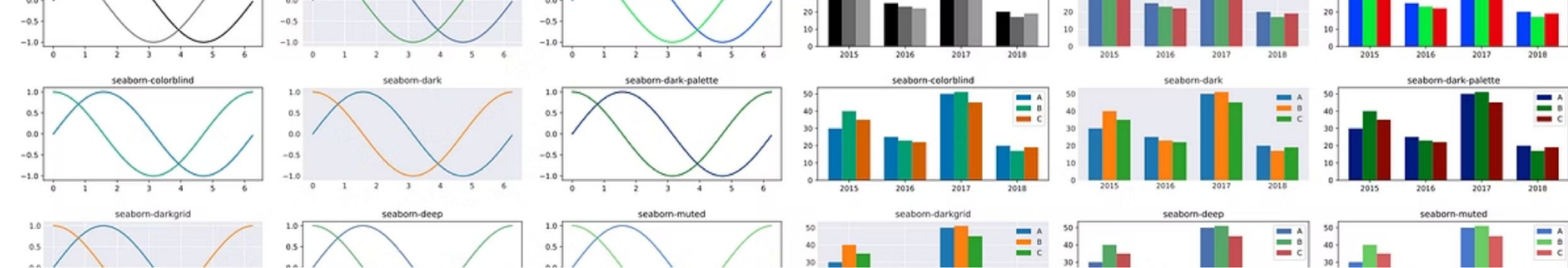
Error	Causa probable	Sugerencia de solución
Atributos mal aplicados	Llamar set_title() fuera de ax	Usar ax.set_title() correctamente
Varios gráficos no independientes	Reutilizar la misma figura	Crear una figura nueva por cada visualización
Sobrecarga de elementos en un solo eje	Falta de modularidad	Dividir en funciones o usar subgráficos

Transición a subgráficos

Ahora que comprendemos cómo construir gráficos con objetos Figure y Axes, estamos listos para dar el siguiente paso: aprender a organizar varios gráficos dentro de una misma figura, utilizando subgráficos.

Esta técnica es clave para representar comparaciones simultáneas o construir dashboards analíticos.





Figuras y subgráficos (subplots)

¿Cómo presentar múltiples visualizaciones dentro de una misma imagen para facilitar análisis simultáneos?

En escenarios reales, a menudo necesitamos mostrar varios gráficos en paralelo, por ejemplo, para comparar métricas entre distintas regiones, productos o periodos. En estos casos, Matplotlib ofrece una herramienta clave: los subgráficos o subplots, que permiten organizar múltiples representaciones visuales en una misma figura.

Trabajar con subgráficos es esencial para crear reportes comparativos, dashboards analíticos o plantillas gráficas reutilizables. Además, aporta claridad y coherencia a la presentación de datos.

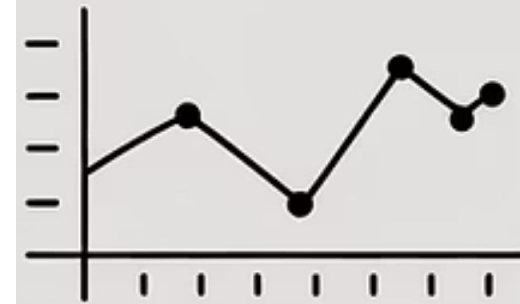
Fundamentos de subplots()

La forma más común y flexible de trabajar con subgráficos es mediante la función `plt.subplots()`, que permite definir el número de filas y columnas, así como personalizar la disposición y dimensiones de los ejes.

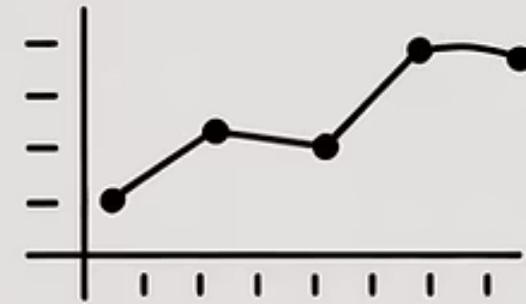
```
fig, axs = plt.subplots(nrows=2, ncols=2, figsize=(10, 8))
```

Esto crea una figura con 4 ejes distribuidos en una grilla de 2 filas y 2 columnas, accesibles a través del arreglo `axs`.

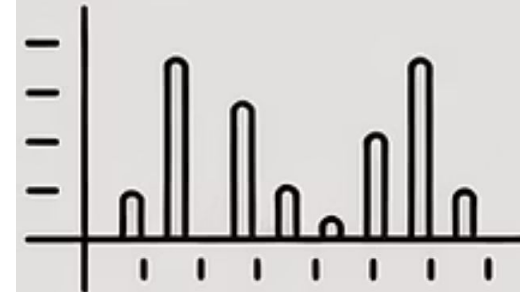
Matplotlib



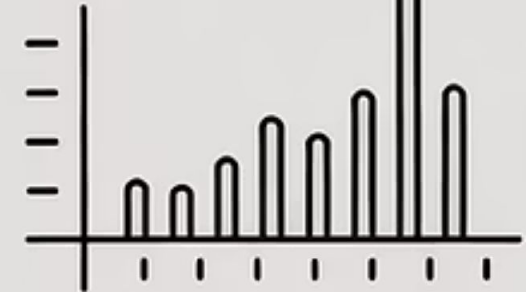
Matplotlib



Matplotlib

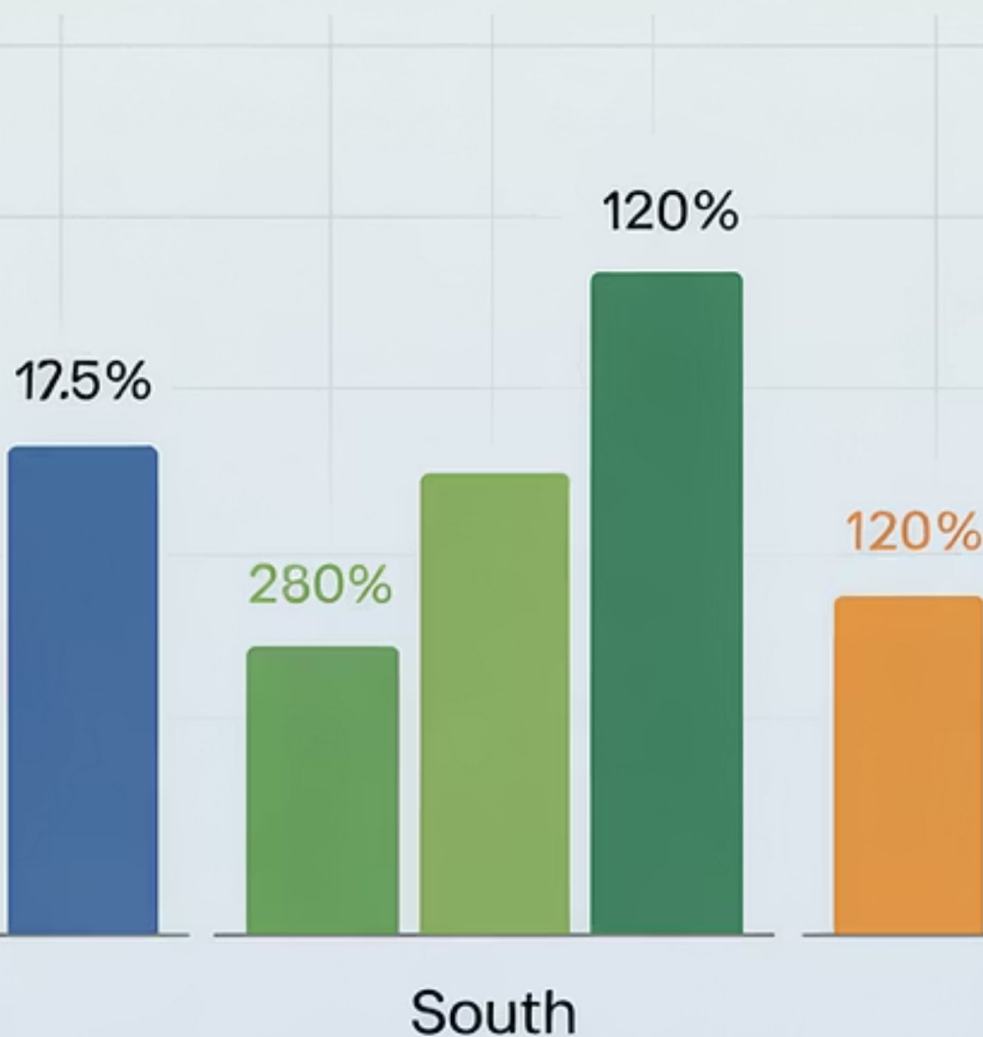


Matplotlib



Regional Sales Comparison

Using Matplotlib Subplots



Ejemplo aplicado: Comparación regional de ventas

{desafío}
latam_

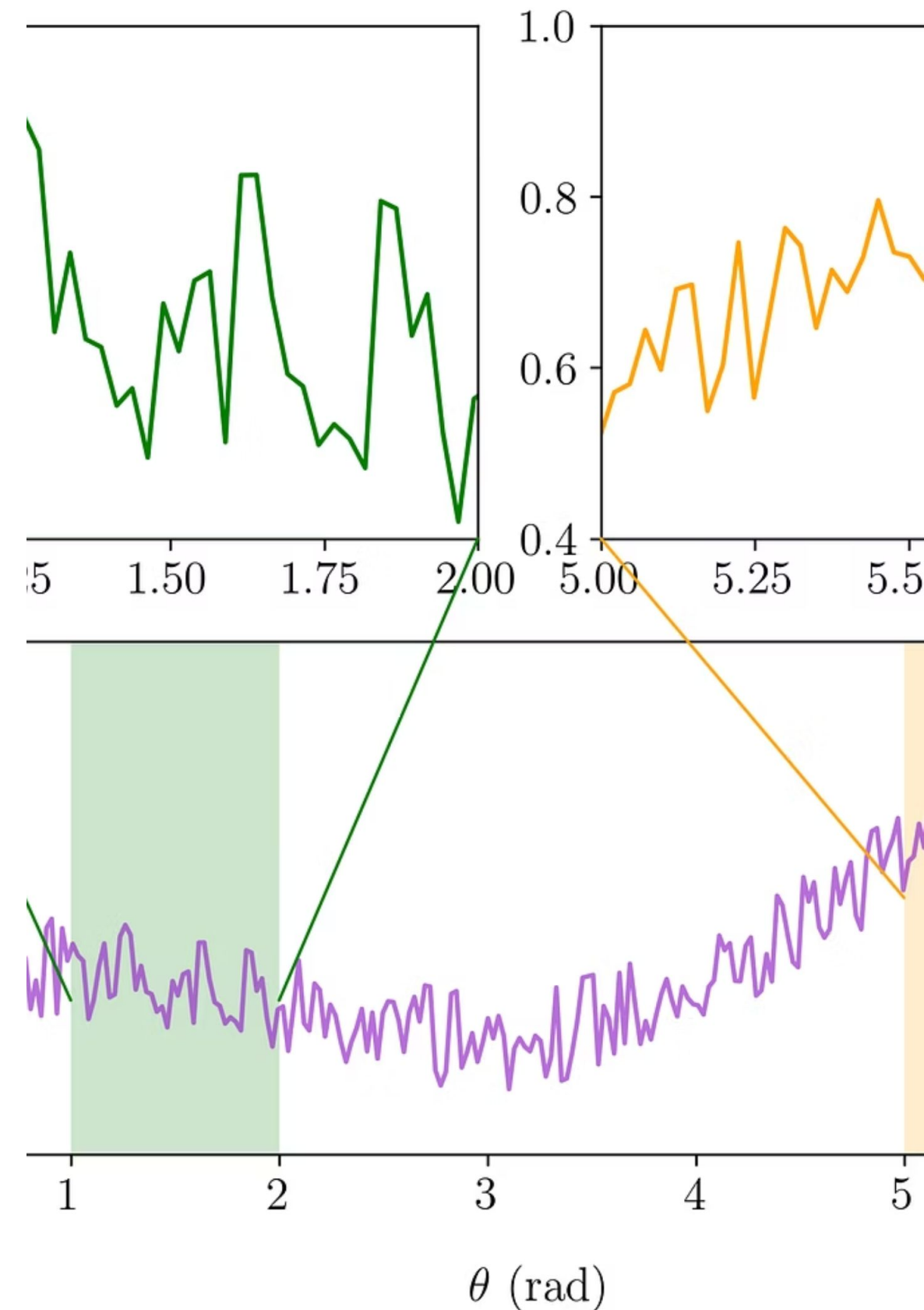
```
import matplotlib.pyplot as plt
meses = ['Ene', 'Feb', 'Mar']
datos = {
    'Norte': [250, 260, 270],
    'Centro': [300, 310, 320],
    'Sur': [200, 190, 210],
    'Austral': [180, 170, 160]
}
fig, axs = plt.subplots(2, 2, figsize=(12, 8))
regiones = list(datos.keys())
for i, ax in enumerate(axs.flat):
    region = regiones[i]
    ax.plot(meses, datos[region], marker='o', color='navy')
    ax.set_title(f'Región {region}')
    ax.set_xlabel('Mes')
    ax.set_ylabel('Ventas ($)')
    ax.grid(True)
fig.suptitle('Comparativo de ventas trimestrales por región')
plt.tight_layout()
plt.subplots_adjust(top=0.9)
plt.show()
```

Buenas prácticas en el uso de subgráficos

Usar `figsize` proporcional al número de subgráficos: evita que los ejes y etiquetas se compriman.

Incluir títulos por subgráfico y uno general con `fig.suptitle()`: mejora la comprensión de cada segmento y del conjunto.

Aplicar `tight_layout()` y `plt.subplots_adjust()` para prevenir solapamientos: clave cuando se utilizan etiquetas extensas.



Errores comunes y recomendaciones

Error	Causa probable	Solución sugerida
Índices incorrectos al usar <code>axs[i]</code>	Arreglo 2D sin recorrer con <code>.flat</code>	Usar <code>axs.flat</code> para iterar linealmente los ejes
Etiquetas y títulos se superponen	Falta de ajuste en el diseño	Aplicar <code>tight_layout()</code> y ajustar márgenes
Pérdida de proporción visual	Tamaño de figura inadecuado	Ajustar <code>figsize</code> y cantidad de filas/columnas

Cuándo usar subgráficos

Comparación visual

Comparar visualmente distintas dimensiones de un mismo fenómeno.

Dashboards

Organizar dashboards en notebooks o presentaciones.

Optimización de espacio

Ahorrar espacio visual sin perder información clave.

El uso de subgráficos en Matplotlib permite organizar visualizaciones múltiples de forma coherente y eficiente, facilitando la comparación simultánea de distintos conjuntos de datos. Dominar esta técnica es clave para construir reportes visuales profesionales, especialmente en contextos donde se analiza información segmentada o se presentan dashboards.

Actividad guiada: Reporte visual comparativo de ventas regionales



Ejercicio

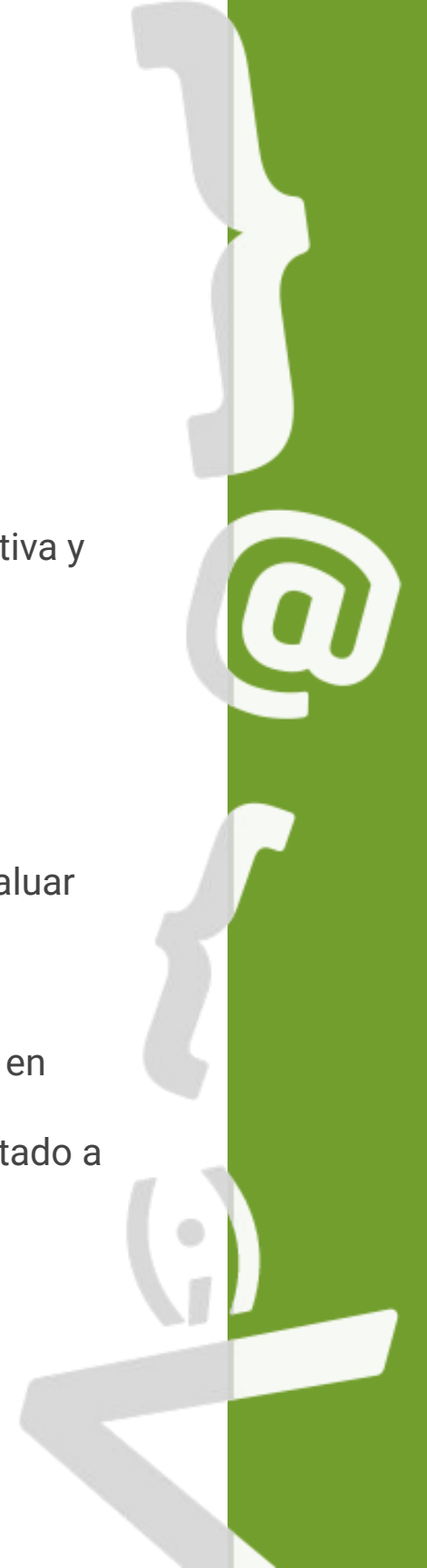
Explorando patrones de uso con visualizaciones en Seaborn

Objetivo de la actividad

Aplicar conocimientos sobre pyplot, el modelo orientado a objetos y subgráficos en Matplotlib para construir una visualización comparativa y profesional de datos segmentados por región, emulando un requerimiento de visualización ejecutiva en entornos productivos reales.

Situación problema

Eres parte del equipo de inteligencia comercial de una empresa de distribución nacional. La gerencia necesita un informe visual para evaluar el comportamiento mensual de ventas en cuatro regiones: Norte, Centro, Sur y Austral. El equipo desea un gráfico comparativo claro y funcional que pueda actualizarse automáticamente con los datos del próximo trimestre. Para ello, se te encarga desarrollar una función en Python que genere un gráfico reutilizable con subgráficos organizados, siguiendo buenas prácticas visuales y aplicando el modelo orientado a objetos.



Ejercicio

Instrucciones

1. Crea un script en Python o una celda de notebook donde:
 1. Construyas una función `reporte_regional(datos)` que reciba un diccionario con ventas mensuales por región.
 2. Utilices `plt.subplots()` para crear una figura con 4 subgráficos (2x2).
 3. Personalices cada eje (`ax`) con títulos, etiquetas y grilla.
 4. Agregues un título general con `fig.suptitle()` y ajustes de diseño (`tight_layout`, `subplots_adjust`).
2. Prueba la función usando un diccionario de datos simulado.
3. Guarda o exporta el gráfico si se desea presentar.



Ejercicio

Instrucciones

Código base sugerido para comenzar

{desafío}
latam_

```
import matplotlib.pyplot as plt

def reporte_regional(datos):
    regiones = list(datos.keys())
    fig, axs = plt.subplots(2, 2, figsize=(12, 8))

    for i, ax in enumerate(axs.flat):
        region = regiones[i]
        meses = list(datos[region].keys())
        ventas = list(datos[region].values())

        ax.plot(meses, ventas, marker='o', color='steelblue')
        ax.set_title(f'Región {region}')
        ax.set_xlabel('Mes')
        ax.set_ylabel('Ventas ($)')
        ax.grid(True)

    fig.suptitle('Informe mensual de ventas por región')
    plt.tight_layout()
    plt.subplots_adjust(top=0.9)
    plt.show()

# Simulación de datos
datos_ventas = {
    'Norte': {'Ene': 240, 'Feb': 260, 'Mar': 250},
    'Centro': {'Ene': 310, 'Feb': 320, 'Mar': 330},
    'Sur': {'Ene': 190, 'Feb': 200, 'Mar': 180},
    'Austral': {'Ene': 170, 'Feb': 160, 'Mar': 175}
}

reporte_regional(datos_ventas)
```

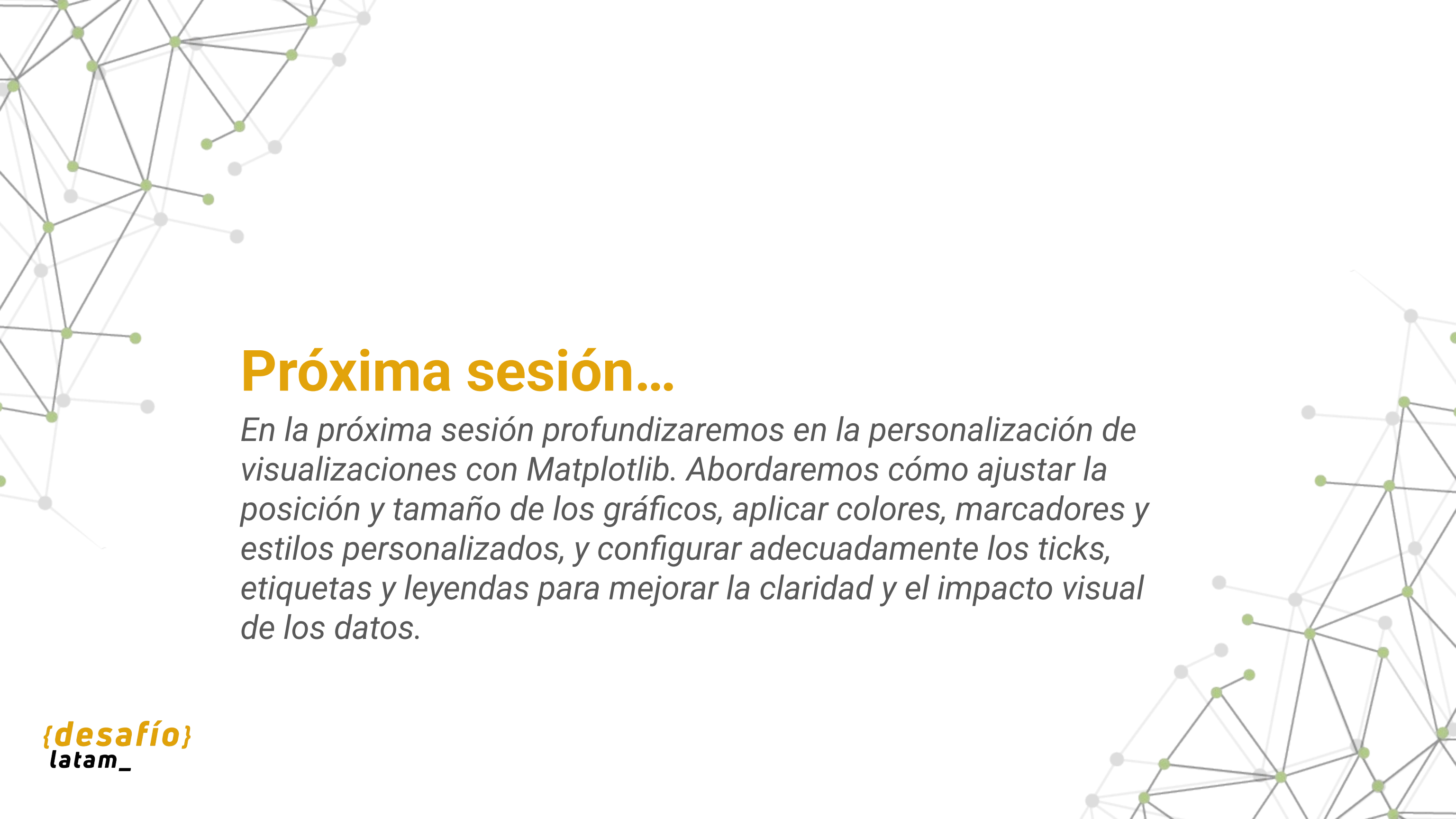
Resumen

Comprendimos las diferencias entre el enfoque funcional (pyplot) y el modelo orientado a objetos (OO) en Matplotlib.

Aprendimos a crear figuras, ejes y elementos gráficos usando objetos Figure y Axes.

Implementamos visualizaciones con múltiples subgráficos utilizando `plt.subplots()`.

Aplicamos buenas prácticas para crear gráficos claros, escalables y orientados a entornos productivos.



Próxima sesión...

En la próxima sesión profundizaremos en la personalización de visualizaciones con Matplotlib. Abordaremos cómo ajustar la posición y tamaño de los gráficos, aplicar colores, marcadores y estilos personalizados, y configurar adecuadamente los ticks, etiquetas y leyendas para mejorar la claridad y el impacto visual de los datos.

Preguntas de cierre

1

¿Qué ventajas ofrece el modelo
orientado a objetos frente a pyplot
cuando se trabaja con visualizaciones
complejas?

2

¿Cuál es la estructura jerárquica de un
gráfico en Matplotlib y qué rol cumple
cada componente?

3

¿Cómo se utilizan los subgráficos
(subplots) para comparar diferentes
conjuntos de datos?

4

¿Qué errores comunes debes evitar al crear múltiples
subgráficos en una misma figura?

5

¿Cómo puedes reutilizar una visualización profesional en
distintos contextos productivos?